

Project Overview & Mathematical Formulation

Project Overview

This project explores how the performance of a two-dimensional heat-diffusion simulation changes when using different execution strategies such as sequential CPU execution, blocking MPI, non-blocking MPI, explicit SIMD, GPU execution (Metal), and a hybrid MPI + SIMD implementation.

The simulated plate begins at a uniform temperature of 20 degrees with a single hot point of 100 degrees at the centre. Heat then diffuses across the plate over a fixed number of timesteps/iterations while the outer boundaries remain fixed at 20 degrees. All configurable parameters such as the tart plate temperature, time steps, hot point position, hot point temp, etc are keep the same across implementations so we can focus on only how the execution method changes performance.

Mathematical Formulation

The simulation is based on the two-dimensional heat equation:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

where:

- $u(x, y, t)$ is the temperature at position (x, y) and time t
- α is the thermal diffusivity
- $\frac{\partial^2 u}{\partial x^2}$ and $\frac{\partial^2 u}{\partial y^2}$ describe the spatial curvature of the temperature field

The spatial second derivatives are approximated using centred finite differences:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}$$

The time derivative is approximated using a forward difference:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t}$$

Substituting these approximations into the heat equation gives the explicit update used in the program:

$$u_{i,j}^{n+1} = u_{i,j}^n + \frac{\alpha \Delta t}{h^2} (u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n)$$

Defining

$$r = \frac{\alpha \Delta t}{h^2}$$

simplifies the implementation to:

$$u_{i,j}^{n+1} = u_{i,j}^n + r(u_{\text{up}} + u_{\text{down}} + u_{\text{left}} + u_{\text{right}} - 4u_{\text{centre}})$$

This shows how the temperature of any cell depends directly on its neighbours and this formula is the main numerical operation performed in every implementation. This allows for the natural question "how can MPI processes exchange the information needed to calculate the new temperature at this cell?". All implementation uses two separate buffers so that all values at timestep $n + 1$ are calculated only from values in timestep n .

Hardware & System

- Model Name: Macbook Air.
- CPU: Apple M3. 4 performance cores, 4 efficiency cores.
- GPU: 8-Core Apple M3 GPU
- Metal Support: Metal 4
- RAM: 8GB
- OS: MacOS Tahoe 26.6.2

Compile Commands

Sequential

```
/opt/homebrew/bin/g++-16 -std=c++20 -O3 heat_timed_seq.cpp -o  
heat_timed_seq
```

Sequential - Auto-Vectorization Disabled

```
/opt/homebrew/bin/g++-16 -std=c++20 -O3 -fno-tree-vectorize  
.heat_timed_seq_novec_tmp.cpp -o heat_timed_seq_novec
```

Explicit SIMD

```
/opt/homebrew/bin/g++-16 -std=c++20 -O3 heat_timed_simd.cpp -o  
heat_timed_simd
```

Blocking MPI

```
/opt/homebrew/bin/g++-16 -std=c++20 -O3 $(mpic++ --showme:compile)  
heat_timed_blocking_mpi.cpp $(mpic++ --showme:link) -o
```

```
heat_timed_blocking_mpi
```

Non-Blocking MPI

```
/opt/homebrew/bin/g++-16 -std=c++20 -O3 $(mpic++ --showme:compile)
heat_timed_nonblocking_mpi.cpp $(mpic++ --showme:link) -o
heat_timed_nonblocking_mpi
```

MPI + SIMD

```
/opt/homebrew/bin/g++-16 -std=c++20 -O3 $(mpic++ --showme:compile)
heat_timed_mpi_simd.cpp $(mpic++ --showme:link) -o heat_timed_mpi_simd
```

Metal

```
xcrun -sdk macosx metal -c heat_kernel.metal -o heat_kernel.air
xcrun -sdk macosx metallib heat_kernel.air -o heat_kernel.metallib
clang++ -std=c++20 -O3 heat_timed_metal.mm -framework Foundation -framework
Metal -o heat_timed_metal
```

Execution Commands

Sequential

```
./heat_timed_seq 3000 3000 1000
./heat_timed_seq 5000 5000 1000
./heat_timed_seq 6000 6000 1000
```

Sequential - Auto-Vectorization Disabled

```
./heat_timed_seq_novec 3000 3000 1000
./heat_timed_seq_novec 5000 5000 1000
./heat_timed_seq_novec 6000 6000 1000
```

Explicit SIMD

```
./heat_timed_simd 3000 3000 1000  
./heat_timed_simd 5000 5000 1000  
./heat_timed_simd 6000 6000 1000
```

Metal

```
./heat_timed_metal 3000 3000 1000  
./heat_timed_metal 5000 5000 1000  
./heat_timed_metal 6000 6000 1000
```

Blocking MPI - 1 Rank

```
mpirun -np 1 ./heat_timed_blocking_mpi 3000 3000 1000  
mpirun -np 1 ./heat_timed_blocking_mpi 5000 5000 1000  
mpirun -np 1 ./heat_timed_blocking_mpi 6000 6000 1000
```

Blocking MPI - 3 Ranks

```
mpirun -np 3 ./heat_timed_blocking_mpi 3000 3000 1000  
mpirun -np 3 ./heat_timed_blocking_mpi 5000 5000 1000  
mpirun -np 3 ./heat_timed_blocking_mpi 6000 6000 1000
```

Blocking MPI - 6 Ranks

```
mpirun -np 6 ./heat_timed_blocking_mpi 3000 3000 1000  
mpirun -np 6 ./heat_timed_blocking_mpi 5000 5000 1000  
mpirun -np 6 ./heat_timed_blocking_mpi 6000 6000 1000
```

Non-Blocking MPI - 1 Rank

```
mpirun -np 1 ./heat_timed_nonblocking_mpi 3000 3000 1000  
mpirun -np 1 ./heat_timed_nonblocking_mpi 5000 5000 1000  
mpirun -np 1 ./heat_timed_nonblocking_mpi 6000 6000 1000
```

Non-Blocking MPI - 3 Ranks

```
mpirun -np 3 ./heat_timed_nonblocking_mpi 3000 3000 1000
mpirun -np 3 ./heat_timed_nonblocking_mpi 5000 5000 1000
mpirun -np 3 ./heat_timed_nonblocking_mpi 6000 6000 1000
```

Non-Blocking MPI - 6 Ranks

```
mpirun -np 6 ./heat_timed_nonblocking_mpi 3000 3000 1000
mpirun -np 6 ./heat_timed_nonblocking_mpi 5000 5000 1000
mpirun -np 6 ./heat_timed_nonblocking_mpi 6000 6000 1000
```

MPI + SIMD - 1 Rank

```
mpirun -np 1 ./heat_timed_mpi_simd 3000 3000 1000
mpirun -np 1 ./heat_timed_mpi_simd 5000 5000 1000
mpirun -np 1 ./heat_timed_mpi_simd 6000 6000 1000
```

MPI + SIMD - 3 Ranks

```
mpirun -np 3 ./heat_timed_mpi_simd 3000 3000 1000
mpirun -np 3 ./heat_timed_mpi_simd 5000 5000 1000
mpirun -np 3 ./heat_timed_mpi_simd 6000 6000 1000
```

MPI + SIMD - 6 Ranks

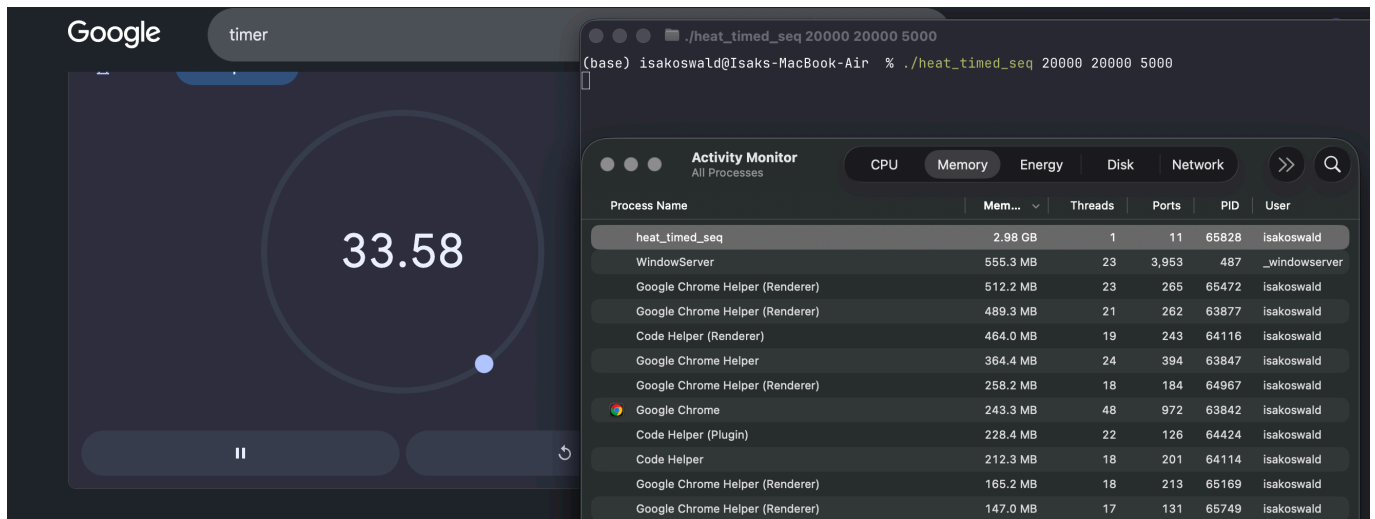
```
mpirun -np 6 ./heat_timed_mpi_simd 3000 3000 1000
mpirun -np 6 ./heat_timed_mpi_simd 5000 5000 1000
mpirun -np 6 ./heat_timed_mpi_simd 6000 6000 1000
```

Stress Test

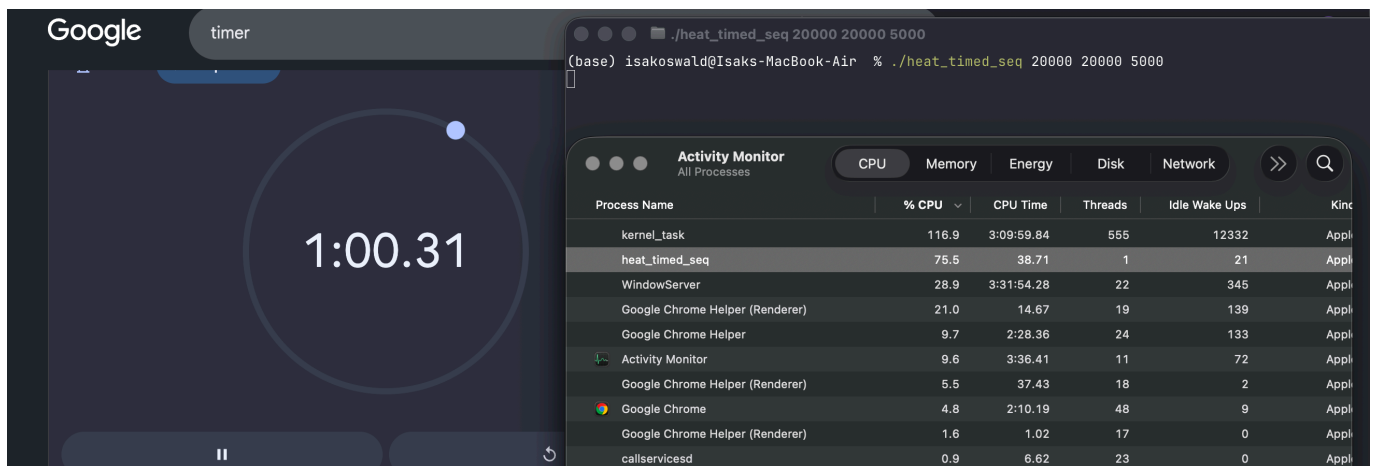
For the stress test, I ran the following configuration on the sequential version:

- Size: 20000 x 20000
- TimeSteps: 5000

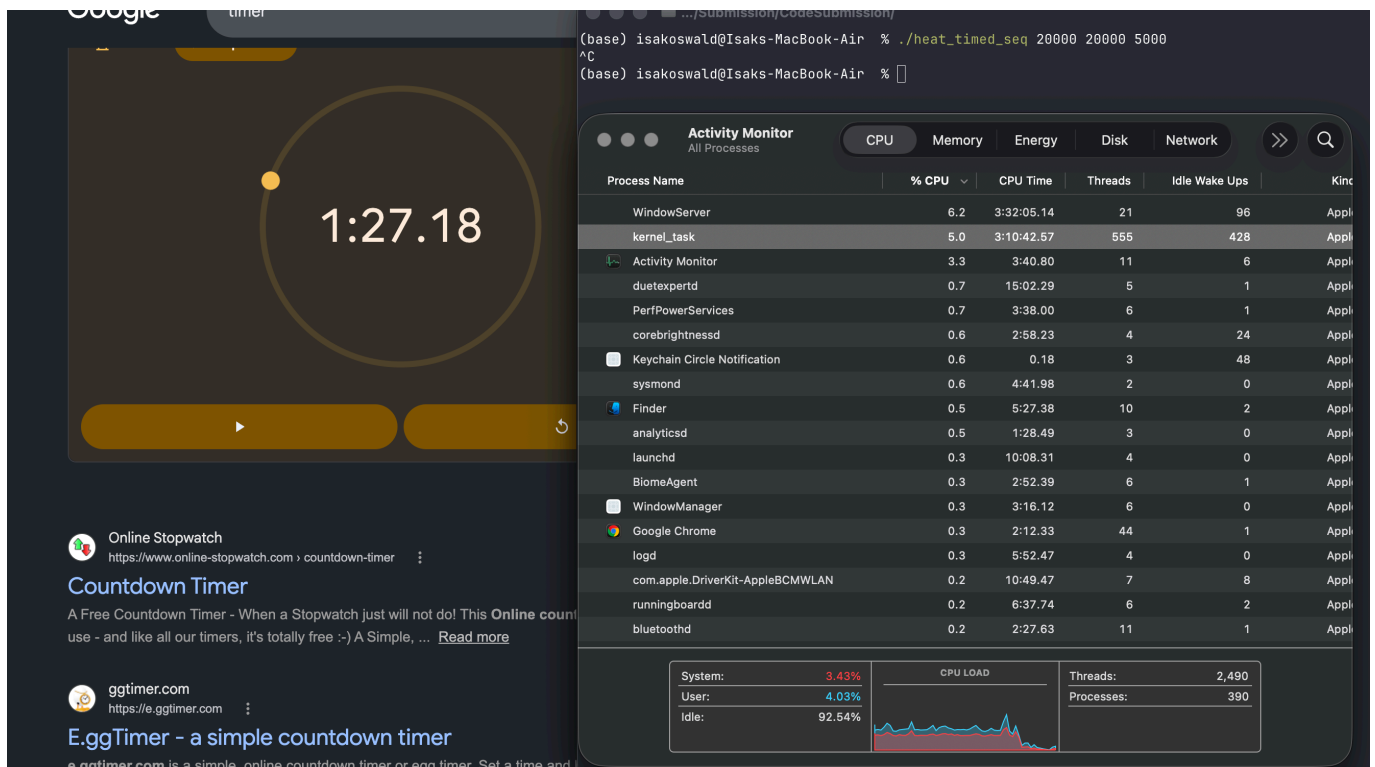
After starting execution, I could see that it was using around three of eight GB of total available RAM.



We can also see that it starts to use significant CPU resources.



After around 1:30 seconds, I recorded a **DNF by user interruption**. After terminating the program, we can see CPU usage drop back down while memory pressure decreased.



Performance Evaluation

1. Evaluation Methodology

I tested the performance of the two-dimensional heat diffusion simulation across several different execution strategies. These included of 7 main types:

- Sequential CPU execution
- Sequential CPU execution with compiler auto-vectorization disabled
- Explicit SIMD execution
- Blocking MPI
- Non-blocking MPI
- Metal GPU execution
- Hybrid MPI + SIMD execution

I decided to test three increasing workload sizes:

Workload	Grid Size	Timesteps
Small	3000 x 3000	1000
Medium	5000 x 5000	1000
Large	6000 x 6000	1000

The timestep count was held constant at 1000 so that the primary independent workload variable was the number of grid cells.

For the MPI implementations, I evaluated three process/rank counts:

MPI Configuration	Rank Count
Single process	1
Moderate parallelism	3
Higher parallelism	6

I executed each configuration twice, so the primary reported execution time is the mean of the two.

During earlier testings, my results showed that larger workloads created substantial thermal throttling on my system. To avoid this, the final benchmark used a five-minute idle cooldown between tests, and fifteen-minute cooldown between test rounds. This took around 8 hours.

2. Correctness Validation

Before comparing performance, I first checked each implementation for consistency.

All implementations produced the same final centre temperature after 1000 timesteps:

Grid Size	Final Centre Temperature
3000 x 3000	20.6442 °C
5000 x 5000	20.6442 °C
6000 x 6000	20.6442 °C

indicating that the different execution strategies preserved the same numerical result for the tested workloads.

3. Raw Execution Time Results

3.1 Sequential -03

Grid Size	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	1397	1290	1343.5
5000 x 5000	4544	4487	4515.5
6000 x 6000	7015	6922	6968.5

The optimized sequential implementation provides the main CPU baseline used when comparing the other execution strategies.

3.2 Sequential with Automatic Vectorization Disabled

This implementation used the same sequential source code but was compiled with:

```
-03 -fno-tree-vectorize
```

This keeps most compiler optimizations enabled while disabling GCC's automatic loop vectorization. This is so we can test our SIMD implementation later.

Grid Size	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	4122	3897	4009.5
5000 x 5000	11381	11008	11194.5
6000 x 6000	17534	17201	17367.5

The large performance difference between this implementation and the normal -03 build demonstrates that compiler-generated SIMD/vectorization is very important in terms of performance to the sequential baseline.

3.3 Explicit SIMD

Grid Size	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	1389	1313	1351.0
5000 x 5000	4431	4312	4371.5
6000 x 6000	7214	7134	7174.0

The explicit SIMD implementation processes multiple adjacent floating-point cells within each vector instruction.

On my system, the native SIMD width was four `float` values per vector.

Importantly, my explicit SIMD implementation performed very similarly to the optimized sequential version using the compiler flags. This suggests that it may be better to just use the `-O3` flag instead of creating your own vectorisation implementation, unless you need fine-grain control.

3.4 Metal GPU

Grid Size	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	1182	1176	1179.0
5000 x 5000	2787	2786	2786.5
6000 x 6000	3673	3668	3670.5

The Metal implementation maps grid cells onto GPU threads rather than just using the CPU as seen in the previous implementations.

The GPU's performance compared to the CPU implementations increased as the workload became larger and larger. This suggests that the larger grids were better able to amortize GPU command-dispatch overhead and provide enough independent work to utilise the GPU effectively. Overall, Metal/OpenCL is my favourite framework I have used throughout the trimester, and always see great performance benefits when using it.

4. MPI Execution Time Results

4.1 Blocking MPI

Grid Size	Ranks	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	1	1340	1284	1312.0
3000 x 3000	3	690	691	690.5
3000 x 3000	6	937	945	941.0

Grid Size	Ranks	Run 1 (ms)	Run 2 (ms)	Mean (ms)
5000 x 5000	1	4550	4495	4522.5
5000 x 5000	3	2344	2344	2344.0
5000 x 5000	6	2747	2788	2767.5
6000 x 6000	1	7739	6918	7328.5
6000 x 6000	3	3496	3495	3495.5
6000 x 6000	6	3859	3822	3840.5

The blocking MPI implementation showed a substantial improvement when moving from one rank (sequential) to three ranks.

This does not however automatically mean more ranks equal better performance as we actually see that increasing from three ranks to six ranks made execution slower for all three workloads.

Since my OpenMPI implementation is running locally, additional MPI processes compete for the same physical CPU resources, caches and memory, ultimately slowing things down. Communication and synchronization overhead also increase as more ranks are introduced which might explain why we see this slowdown from three to six ranks.

4.2 Non-Blocking MPI

Grid Size	Ranks	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	1	1335	1284	1309.5
3000 x 3000	3	688	691	689.5
3000 x 3000	6	876	881	878.5
5000 x 5000	1	4543	4491	4517.0
5000 x 5000	3	2346	2341	2343.5
5000 x 5000	6	2611	2601	2606.0
6000 x 6000	1	6972	6910	6941.0
6000 x 6000	3	3490	3491	3490.5
6000 x 6000	6	3657	3698	3677.5

The non-blocking implementation followed the same overall scaling behaviour as blocking MPI.

We can see that the three ranks still consistently provided the best execution time. Six ranks remained faster than one rank, but slower than three ranks.

I expected significant speedups compared to the blocking version, but we actually see very small differences at three ranks. I suspect that this is happening because all MPI processes were running on the same physical machine, making halo transfers relatively inexpensive compared with communication

over an external network. It would be interesting to see these programs tested over different physical nodes.

At six ranks, the non-blocking version showed a more noticeable advantage over the blocking version. This suggests that communication/computation overlap becomes more useful when rank count and synchronization overheads increase.

4.3 Hybrid MPI + SIMD

Grid Size	Ranks	Run 1 (ms)	Run 2 (ms)	Mean (ms)
3000 x 3000	1	1855	1821	1838.0
3000 x 3000	3	743	742	742.5
3000 x 3000	6	942	934	938.0
5000 x 5000	1	5667	5498	5582.5
5000 x 5000	3	2486	2481	2483.5
5000 x 5000	6	2723	2654	2688.5
6000 x 6000	1	8588	8295	8441.5
6000 x 6000	3	3789	3764	3776.5
6000 x 6000	6	3851	3880	3865.5

The hybrid implementation demonstrated good scaling relative to its own one-rank baseline. However, its absolute execution time was consistently slower than the simpler MPI only implementations. This shows that combining multiple parallelization techniques does not always promise additive performance gains.

On my hardware, the MPI processes are already competing for shared CPU and memory resources. Adding the explicit SIMD logic also increases computational overheads, but the stencil is mainly dependent on memory access. Therefore, the limiting factor is most likely shared memory bandwidth rather than arithmetic throughput.

From my testing with SIMD, it makes much more sense to just use the appropriate compiler optimisation flags rather than creating your own SIMD implementation. It still was however an insightful learning experience.

5. Overall Mean Execution Time Comparison

Implementation	3000 ² (ms)	5000 ² (ms)	6000 ² (ms)
Sequential -03	1343.5	4515.5	6968.5
Sequential, no auto-vectorization	4009.5	11194.5	17367.5
Explicit SIMD	1351.0	4371.5	7174.0

Implementation	3000 ² (ms)	5000 ² (ms)	6000 ² (ms)
Metal GPU	1179.0	2786.5	3670.5
MPI Blocking, 1 rank	1312.0	4522.5	7328.5
MPI Blocking, 3 ranks	690.5	2344.0	3495.5
MPI Blocking, 6 ranks	941.0	2767.5	3840.5
MPI Non-blocking, 1 rank	1309.5	4517.0	6941.0
MPI Non-blocking, 3 ranks	689.5	2343.5	3490.5
MPI Non-blocking, 6 ranks	878.5	2606.0	3677.5
MPI + SIMD, 1 rank	1838.0	5582.5	8441.5
MPI + SIMD, 3 ranks	742.5	2483.5	3776.5
MPI + SIMD, 6 ranks	938.0	2688.5	3865.5

Across the tested workloads, three-rank non-blocking MPI produced the best execution times.

In saying this, Metal became increasingly more competitive as grid size increased and was close to the MPI implementations for the largest workload. This means that Metal may eventually overtake MPI, but more testing would be needed.

Explicit SIMD was approximately equal to optimized sequential execution, while the non-vectorized sequential implementation was substantially slower than both. This shows that SIMD is useful, but you should probably not code it yourself.

6. Speedup Relative to Optimized Sequential

Speedup relative to the optimized sequential implementation is calculated using:

$$S = \frac{T_{\text{sequential}}}{T_{\text{implementation}}}$$

Implementation	3000 ² Speedup	5000 ² Speedup	6000 ² Speedup
Explicit SIMD	0.99x	1.03x	0.97x
Metal GPU	1.14x	1.62x	1.90x
MPI Blocking, 1 rank	1.02x	1.00x	0.95x
MPI Blocking, 3 ranks	1.95x	1.93x	1.99x
MPI Blocking, 6 ranks	1.43x	1.63x	1.81x
MPI Non-blocking, 1 rank	1.03x	1.00x	1.00x
MPI Non-blocking, 3 ranks	1.95x	1.93x	2.00x

Implementation	3000 ² Speedup	5000 ² Speedup	6000 ² Speedup
MPI Non-blocking, 6 ranks	1.53x	1.73x	1.89x
MPI + SIMD, 1 rank	0.73x	0.81x	0.83x
MPI + SIMD, 3 ranks	1.81x	1.82x	1.85x
MPI + SIMD, 6 ranks	1.43x	1.68x	1.80x

We can see that the best CPU result was produced by the three-rank MPI implementations.

For the largest workload (6000 x 6000), non-blocking MPI reached approximately a

$$\frac{6968.5}{3490.5} \approx 2.00$$

speedup relative to optimized sequential execution.

Metal also scaled strongly with problem size, increasing from approximately **1.14x** speedup for the smallest grid to approximately **1.90x** for the largest grid.

7. MPI Strong Scaling

MPI scaling can also be evaluated relative to the one-rank execution of the same MPI implementation.

This means that the MPI speedup for p ranks is:

$$S_p = \frac{T_1}{T_p}$$

and the parallel efficiency is:

$$E_p = \frac{S_p}{p}$$

An ideal result would produce a speedup equal to the number of ranks and therefore an efficiency of 100%. In practice, communication, synchronization, load imbalance, resource contention and serial work reduce efficiency.

7.1 Blocking MPI Scaling

Grid Size	Ranks	Mean Time (ms)	Speedup vs 1 Rank	Parallel Efficiency
3000 ²	1	1312.0	1.00x	100.0%
3000 ²	3	690.5	1.90x	63.3%
3000 ²	6	941.0	1.39x	23.2%
5000 ²	1	4522.5	1.00x	100.0%
5000 ²	3	2344.0	1.93x	64.3%

Grid Size	Ranks	Mean Time (ms)	Speedup vs 1 Rank	Parallel Efficiency
5000 ²	6	2767.5	1.63x	27.2%
6000 ²	1	7328.5	1.00x	100.0%
6000 ²	3	3495.5	2.10x	69.9%
6000 ²	6	3840.5	1.91x	31.8%

We can see that the best blocking MPI scaling occurred with three ranks on the largest workload.

The largest workload yeilded approximately **2.10x** speedup using three ranks, corresponding to approximately 69.9% parallel efficiency.

However, as we have previously discussed, six ranks were slower than three ranks for every workload. This caused the six-rank efficiency to fall substantially.

The results indicate that three ranks provide a better balance between additional parallel computation and shared-resource contention on the test system. I would expect this result to be significantly different on different hardware, but more testing would be required.

7.2 Non-Blocking MPI Scaling

Grid Size	Ranks	Mean Time (ms)	Speedup vs 1 Rank	Parallel Efficiency
3000 ²	1	1309.5	1.00x	100.0%
3000 ²	3	689.5	1.90x	63.3%
3000 ²	6	878.5	1.49x	24.8%
5000 ²	1	4517.0	1.00x	100.0%
5000 ²	3	2343.5	1.93x	64.2%
5000 ²	6	2606.0	1.73x	28.9%
6000 ²	1	6941.0	1.00x	100.0%
6000 ²	3	3490.5	1.99x	66.3%
6000 ²	6	3677.5	1.89x	31.5%

The non-blocking implementation shows the same overall pattern as the blocking implementation.

Interestingly, we can see that the six-rank non-blocking implementation performs better than the six-rank blocking implementation for every tested workload. This suggests that that computation/communication overlap becomes more valuable as the number of processes increase.

7.3 MPI + SIMD Scaling

Grid Size	Ranks	Mean Time (ms)	Speedup vs 1 Rank	Parallel Efficiency
3000 ²	1	1838.0	1.00x	100.0%
3000 ²	3	742.5	2.48x	82.5%
3000 ²	6	938.0	1.96x	32.7%
5000 ²	1	5582.5	1.00x	100.0%
5000 ²	3	2483.5	2.25x	74.9%
5000 ²	6	2688.5	2.08x	34.6%
6000 ²	1	8441.5	1.00x	100.0%
6000 ²	3	3776.5	2.24x	74.5%
6000 ²	6	3865.5	2.18x	36.4%

The hybrid implementation shows quite strong speedup when measured against its own one-rank implementation.

For example, the 3000 x 3000 case reaches approximately **2.48x** speedup using three ranks.

While this seems good, we must rememeber that the hybrid implementation is slower then the standard MPI implementation, so a large relative speedup does not mean that the MPI + SIMD implemnetation is better than the standard MPI implementation.

8. Blocking vs Non-Blocking MPI

The relative improvement from non-blocking communication was calculated as:

$$\text{Improvement} = \frac{T_{\text{blocking}} - T_{\text{nonblocking}}}{T_{\text{blocking}}} \times 100$$

Positive values indicate that non-blocking MPI was faster.

Grid Size	Ranks	Blocking (ms)	Non-blocking (ms)	Non-blocking Improvement
3000 ²	1	1312.0	1309.5	0.2%
3000 ²	3	690.5	689.5	0.1%
3000 ²	6	941.0	878.5	6.6%
5000 ²	1	4522.5	4517.0	0.1%
5000 ²	3	2344.0	2343.5	~0.0%
5000 ²	6	2767.5	2606.0	5.8%
6000 ²	1	7328.5	6941.0	5.3%
6000 ²	3	3495.5	3490.5	0.1%

Grid Size	Ranks	Blocking (ms)	Non-blocking (ms)	Non-blocking Improvement
6000 ²	6	3840.5	3677.5	4.2%

For three ranks, we can see that both the blocking and non-blocking implementations produced almost identical execution times.

This suggests that the cost of halo communication was already small relative to the amount of computation performed between communication phases.

At six ranks, non-blocking MPI was consistently faster than blocking MPI by approximately 4–7%.

This shows that the overlap strategy may becomes more useful as rank count increases and communication and synchronization overhead become more significant.

9. SIMD Evaluation

9.1 Effect of Compiler Auto-Vectorization

Grid Size	No Auto-Vectorization (ms)	Normal -O3 (ms)	-O3 Speedup
3000 ²	4009.5	1343.5	2.98x
5000 ²	11194.5	4515.5	2.48x
6000 ²	17367.5	6968.5	2.49x

The optimized sequential implementation was approximately **2.5–3.0x** faster than the same code compiled with automatic vectorization disabled.

This shows that GCC's **-O3** optimizer is already generating effective SIMD instructions for the sequential stencil.

Therefore, the normal sequential implementation should not be interpreted as a purely scalar baseline.

9.2 Explicit SIMD vs Non-Vectorized Sequential

Grid Size	Non-Vectorized (ms)	Explicit SIMD (ms)	SIMD Speedup
3000 ²	4009.5	1351.0	2.97x
5000 ²	11194.5	4371.5	2.56x
6000 ²	17367.5	7174.0	2.42x

Explicit SIMD produced a large improvement relative to execution where compiler vectorization was disabled.

The measured speedups ranged from approximately **2.42x** to **2.97x**. This demonstrates that SIMD itself is effective for the heat-diffusion stencil.

9.3 Explicit SIMD vs Optimized Sequential

Grid Size	Sequential -03 (ms)	Explicit SIMD (ms)	Relative Speedup
3000 ²	1343.5	1351.0	0.99x
5000 ²	4515.5	4371.5	1.03x
6000 ²	6968.5	7174.0	0.97x

Explicit SIMD and the normal optimized sequential implementation produced very similar execution times.

At the medium size test case, explicit SIMD was approximately only 3% faster, while at the other two sizes the optimized sequential implementation was slightly faster. This is likely just due to runtime differences rather than one method being better than the other.

10. Metal GPU Evaluation

Grid Size	Sequential -03 (ms)	Metal (ms)	Metal Speedup	Time Reduction
3000 ²	1343.5	1179.0	1.14x	12.2%
5000 ²	4515.5	2786.5	1.62x	38.3%
6000 ²	6968.5	3670.5	1.90x	47.3%

Metal showed the clearest increase in relative performance as the workload became larger.

In the smallest test case, Metal provided only approximately **1.14x** speedup over sequential execution. This is quite a weak result in comparison to my other tasks where I used OpenCL/Metal.

At the medium size, this increased to approximately **1.62x**. Then at the largest size, Metal reached approximately **1.90x** speedup. This trend suggests that larger workloads provide enough parallel work to make better use of the GPU.

11. Best Performing Implementation

Grid Size	Fastest Implementation	Mean Time	Speedup vs Sequential
3000 ²	MPI Non-blocking, 3 ranks	689.5 ms	1.95x
5000 ²	MPI Non-blocking, 3 ranks	2343.5 ms	1.93x
6000 ²	MPI Non-blocking, 3 ranks	3490.5 ms	2.00x

Three-rank non-blocking MPI produced the fastest average execution time for every tested workload. But gap between MPI and Metal decreases as the workload size increases.

For 6000 x 6000:

```
MPI Non-blocking, 3 ranks = 3490.5 ms
Metal GPU                  = 3670.5 ms
```

The difference is approximately:

$$\frac{3670.5 - 3490.5}{3490.5} \times 100 \approx 5.2\%$$

so Metal was only around 5.2% slower than the fastest MPI configuration for the largest workload.

12. Effect of Increasing MPI Rank Count

Grid Size	Implementation	3 Ranks (ms)	6 Ranks (ms)	Effect of 6 Ranks
3000 ²	Blocking MPI	690.5	941.0	36.3% slower
3000 ²	Non-blocking MPI	689.5	878.5	27.4% slower
3000 ²	MPI + SIMD	742.5	938.0	26.3% slower
5000 ²	Blocking MPI	2344.0	2767.5	18.1% slower
5000 ²	Non-blocking MPI	2343.5	2606.0	11.2% slower
5000 ²	MPI + SIMD	2483.5	2688.5	8.3% slower
6000 ²	Blocking MPI	3495.5	3840.5	9.9% slower
6000 ²	Non-blocking MPI	3490.5	3677.5	5.4% slower
6000 ²	MPI + SIMD	3776.5	3865.5	2.4% slower

Increasing rank count from three to six would often reduce performance.

But interestingly, we can see that the penalty became smaller as workload size increased.

For blocking MPI, six ranks were approximately 36.3% slower than three ranks for the 3000 x 3000 workload, but only approximately 9.9% slower at 6000 x 6000 size. A similar trend happens in the non-blocking and hybrid implementations.

This makes me think that larger problem sizes improve the computation-to-overhead ratio. Each process receives more useful work, allowing the extra communication and process-management overhead of six ranks to become smaller compared to the computation workload.

While the above is true, six ranks still never became faster than three ranks within the tested workload range.

13. Hybrid MPI + SIMD Evaluation

The hybrid MPI + SIMD implementation was designed to combine both process-level parallelism and data-level parallelism.

Even though this implementation combines multiple optimization strategies, it would be intuitive to assume it would be the fastest. However, the hybrid version did not outperform plain MPI.

For example, at 5000 x 5000 with three ranks:

```
MPI Blocking      = 2344.0 ms
MPI Non-blocking  = 2343.5 ms
MPI + SIMD        = 2483.5 ms
```

At 6000 x 6000 with three ranks:

```
MPI Blocking      = 3495.5 ms
MPI Non-blocking  = 3490.5 ms
MPI + SIMD        = 3776.5 ms
```

This shows that parallel optimizations are not naturally additive.

There are two main factors that can explain the result:

First, the standard MPI implementation is compiled with `-O3`, meaning that the inner loop may already be vectorised by the compiler.

Second, multiple MPI processes are simultaneously reading and writing large contiguous arrays. The processes therefore compete for the same memory subsystem.

16. Limitations

While I have captured a variety of different results, some key limitations should be considered.

First, all MPI testing was performed on a single physical machine. Since MPI is commonly used across multiple different physical nodes, communication characteristics may differ substantially from shared-memory process communication. This means that the results from running the program may be different depending on how/where MPI is running.

Third, only two benchmark repetitions were used whereas I would like to use 5-10. This was a deliberate trade-off as extensive repeated testing caused thermal throttling making the quality of results worse as time went on. I used long cooling intervals to improve the quality of the two measurements, but the two measurements already took around eight hours.

Fourth, final-centre temperature was used as the primary numerical consistency check. A stronger future validation method may include of a checksum.

Finally, compiler optimization has a substantial influence on CPU performance. The explicit SIMD implementation used GCC's experimental SIMD support, while the Metal implementation used Apple's Metal toolchain. Therefore we cannot really be sure what optimisations the compiler is making for each implementation meaning that platform differences cannot be completely eliminated.

17. Key Findings

The evaluation produced several important findings.

1. **Three MPI ranks provided the best overall execution time.**
2. **Increasing MPI process count did not guarantee further performance improvement.**
3. **Non-blocking MPI produced only a modest improvement at low rank counts.**
4. **SIMD provided a major benefit relative to genuinely scalar execution.**
5. **The optimized sequential compiler already exploited SIMD.**
6. **Metal became increasingly effective as problem size increased.**
7. **The hybrid MPI + SIMD implementation did not provide additive speedup.**
8. **Larger workloads reduced the relative penalty of additional MPI ranks.**
9. **The final benchmark methodology produced highly repeatable measurements.**
10. **All tested implementations produced the same final centre temperature.**

Level 3 Discussion

I think that this project goes beyond a basic parallel implementation by combining several concurrent and distributed programming techniques and evaluating them comparatively rather than implementing only one single approach.

The important added work seen in my project includes:

- **Uneven MPI domain decomposition:** Allows grid rows to be distributed fairly even when the number of rows is not evenly divisible by the number of MPI processes.
- **Blocking and non-blocking MPI implementations:** Allowing communication strategies to be compared directly.
- **Explicit SIMD vectorization:** Allowing multiple adjacent grid cells to be processed within a single vector operation.
- **Hybrid MPI + SIMD parallelism:** Combining process-level parallelism with data-level parallelism.

- **Metal GPU acceleration:** Mapping the same finite-difference stencil onto a two-dimensional GPU thread grid.
- **Stronger benchmarking methodology:** I include multiple increasing workloads, repeated measurements, MPI scaling analysis, speedup and efficiency calculations, compiler auto-vectorization comparison, and cooldown periods to reduce thermal-throttling during testing.

Below you can see successful execution of the correctness test program.

```

2D Heat Diffusion Demo
Grid: 20 x 20      Step: 40 / 40

  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.1   20.2   20.1   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.1   20.6   21.6   20.6   20.1   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.1   20.6   23.1   28.2   23.1   20.6   20.1   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.2   21.6   28.2   41.6   28.2   21.6   20.2   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.1   20.6   23.1   28.2   23.1   20.6   20.1   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.1   20.6   21.6   20.6   20.1   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.1   20.2   20.1   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0
  20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0   20.0

Centre temperature: 41.56 C

Demo complete.
(base) isakoswald@Isaks-MacBook-Air %

```

Log Files

The two log files, results.txt and benchmark_console.log are shown below:

```
benchmark_console.... results.txt
Repeat 1 | Sequential O3 | 3000x3000 | 100w steps
Command: ./heat_timed_seq 3000 3000 1000
Started: Fri 11 Sep 2026 02:08:13 AEST
Sequential heat diffusion complete.
Plate: 3000 x 3000
Timesteps: 1000
Final centre temperature: 20.6442 C
Execution time: 1397 ms
Finished: Fri 11 Sep 2026 02:08:15 AEST
Exit code: 0
=====
RUN 2 / 78
Repeat 1 | Sequential O3 No Auto-Vectorization | 3000x3000 | 1000 steps
Command: ./heat_timed_seq_novec 3000 3000 1000
Started: Fri 11 Sep 2026 02:13:15 AEST
Sequential heat diffusion complete.
Plate: 3000 x 3000
Timesteps: 1000
Final centre temperature: 20.6442 C
Execution time: 4122 ms
Finished: Fri 11 Sep 2026 02:13:20 AEST
Exit code: 0
=====
RUN 3 / 78
Repeat 1 | Explicit SIMD | 3000x3000 | 1000 steps
Command: ./heat_timed_simd 3000 3000 1000
Started: Fri 11 Sep 2026 02:18:20 AEST
SIMD width: 4 floats per vector
SIMD heat diffusion complete.
Plate: 3000 x 3000
Timesteps: 1000
Final centre temperature: 20.6442 C
Execution time: 1389 ms
Finished: Fri 11 Sep 2026 02:18:22 AEST
Exit code: 0
=====
RUN 4 / 78
Repeat 1 | Metal | 3000x3000 | 1000 steps
Command: ./heat_timed_metal 3000 3000 1000
Started: Fri 11 Sep 2026 02:23:22 AEST
Metal device: Apple M3
Metal heat diffusion complete.
Plate: 3000 x 3000
Timesteps: 1000
Final centre temperature: 20.6442 C
Execution time: 1182 ms
Finished: Fri 11 Sep 2026 02:23:23 AEST
Exit code: 0
=====
RUN 5 / 78
Repeat 1 | MPI Blocking | 1 ranks | 3000x3000 | 1000 steps
Command: mpirun -np 1 ./heat_timed_blocking_mpi 3000 3000 1000
Started: Fri 11 Sep 2026 02:28:23 AEST
Blocking MPI heat diffusion complete.
Plate: 3000 x 3000
Timesteps: 1000
Execution time: 1182 ms
Finished: Fri 11 Sep 2026 02:28:24 AEST
Exit code: 0
=====
Sequential Flat Float, 3000x3000, 1000, 1, 100, 20.6442, 1397
Sequential No Auto-Vectorization, 3000x3000, 1000, 1, 100, 20.6442, 4122
SIMD Flat Float, 3000x3000, 1000, 1, 100, 20.6442, 1389
Metal Flat Float, 3000x3000, 1000, 1, 100, 20.6442, 1182
MPI Blocking Flat Float, 3000x3000, 1000, 1, 100, 20.6442, 1340
MPI NonBlocking Flat Float, 3000x3000, 1000, 1, 100, 20.6442, 1335
MPI + SIMD NonBlocking, 3000x3000, 1000, 1, 100, 20.6442, 1353
MPI Blocking Flat Float, 3000x3000, 1000, 3, 100, 20.6442, 690
MPI NonBlocking Flat Float, 3000x3000, 1000, 3, 100, 20.6442, 688
MPI + SIMD NonBlocking, 3000x3000, 1000, 3, 100, 20.6442, 743
MPI Blocking Flat Float, 3000x3000, 1000, 6, 100, 20.6442, 937
MPI NonBlocking Flat Float, 3000x3000, 1000, 6, 100, 20.6442, 876
MPI + SIMD NonBlocking, 3000x3000, 1000, 6, 100, 20.6442, 942
Sequential Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4544
Sequential No Auto-Vectorization, 5000x5000, 1000, 1, 100, 20.6442, 11381
SIMD Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4431
Metal Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 2787
MPI Blocking Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4550
MPI NonBlocking Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4543
MPI + SIMD NonBlocking, 5000x5000, 1000, 1, 100, 20.6442, 5667
MPI Blocking Flat Float, 5000x5000, 1000, 3, 100, 20.6442, 2344
MPI NonBlocking Flat Float, 5000x5000, 1000, 3, 100, 20.6442, 2346
MPI + SIMD NonBlocking, 5000x5000, 1000, 3, 100, 20.6442, 2486
MPI Blocking Flat Float, 5000x5000, 1000, 6, 100, 20.6442, 2747
MPI NonBlocking Flat Float, 5000x5000, 1000, 6, 100, 20.6442, 2611
MPI + SIMD NonBlocking, 5000x5000, 1000, 6, 100, 20.6442, 2723
Sequential Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 7015
Sequential No Auto-Vectorization, 6000x6000, 1000, 1, 100, 20.6442, 17534
SIMD Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 7214
Metal Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 3673
MPI Blocking Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 7739
MPI NonBlocking Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 6972
MPI + SIMD NonBlocking, 6000x6000, 1000, 1, 100, 20.6442, 8588
MPI Blocking Flat Float, 6000x6000, 1000, 3, 100, 20.6442, 3496
MPI NonBlocking Flat Float, 6000x6000, 1000, 3, 100, 20.6442, 3490
MPI + SIMD NonBlocking, 6000x6000, 1000, 3, 100, 20.6442, 3789
MPI Blocking Flat Float, 6000x6000, 1000, 6, 100, 20.6442, 3859
MPI NonBlocking Flat Float, 6000x6000, 1000, 6, 100, 20.6442, 3657
MPI + SIMD NonBlocking, 6000x6000, 1000, 6, 100, 20.6442, 3851
MPI + SIMD NonBlocking, 6000x6000, 1000, 6, 100, 20.6442, 3880
MPI NonBlocking Flat Float, 6000x6000, 1000, 6, 100, 20.6442, 3698
MPI Blocking Flat Float, 6000x6000, 1000, 6, 100, 20.6442, 3822
MPI + SIMD NonBlocking, 6000x6000, 1000, 3, 100, 20.6442, 3764
MPI NonBlocking Flat Float, 6000x6000, 1000, 3, 100, 20.6442, 3491
MPI Blocking Flat Float, 6000x6000, 1000, 3, 100, 20.6442, 3495
MPI + SIMD NonBlocking, 6000x6000, 1000, 1, 100, 20.6442, 8395
MPI NonBlocking Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 6910
MPI Blocking Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 6918
Metal Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 3688
SIMD Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 7154
Sequential No Auto-Vectorization, 6000x6000, 1000, 1, 100, 20.6442, 17201
Sequential Flat Float, 6000x6000, 1000, 1, 100, 20.6442, 6922
MPI + SIMD NonBlocking, 5000x5000, 1000, 6, 100, 20.6442, 2654
MPI NonBlocking Flat Float, 5000x5000, 1000, 6, 100, 20.6442, 2601
MPI Blocking Flat Float, 5000x5000, 1000, 6, 100, 20.6442, 2788
MPI + SIMD NonBlocking, 5000x5000, 1000, 3, 100, 20.6442, 2481
MPI NonBlocking Flat Float, 5000x5000, 1000, 3, 100, 20.6442, 2341
MPI Blocking Flat Float, 5000x5000, 1000, 3, 100, 20.6442, 2344
MPI + SIMD NonBlocking, 5000x5000, 1000, 1, 100, 20.6442, 5498
MPI NonBlocking Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4491
MPI Blocking Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4495
Metal Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 2786
SIMD Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4312
Sequential No Auto-Vectorization, 5000x5000, 1000, 1, 100, 20.6442, 11008
Sequential Flat Float, 5000x5000, 1000, 1, 100, 20.6442, 4487
```